Experiment 02 – Datatypes and Operators

Aim: To understand the data types, scope and lifetime of variables in a program. Also understand the significance of type conversions in C and the order of precedence in which the operations could be performed.

Tools/Software Required: Turbo C++ / Dev C++ / VSCode + MingGW Compiler

Theory:

A variable is just a named area of storage that can hold a single value (numeric or character). The C language demands that you declare the name of each variable that you are going to use and its type, or class, before you actually try to do anything with it. The Programming language C has two main variable types

- Local Variables:
- Global Variables:

Local Variables:
Local variables scope is confined within the block or function where it is defined. Local variables must always be defined at the top of a block. When a local variable is defined - it is not initialized by the system, you must initialize it yourself.
When execution of the block starts the variable is available, and when the block ends the variable 'dies'.

Global Variables:
Global variable is defined at the top of the program file and it can be visible and modified by any function that may reference it. Global variables are initialized automatically by the system when you define them!

Data types:
Data types specify how we enter data into our programs and what type of data we enter. C language has some predefined set of data types to handle various kinds of data that we can use in our program. These data types have different storage capacities.
C language supports 2 different type of data types:

Primary data types:
These are fundamental data types in C namely integer (int), floating point (float), character (char) and void.

Derived data types:
Derived data types are nothing but primary data types but a little twisted or grouped together like array, stucture, union and pointer.
Type Casting:

Type casting refers to changing a variable of one data type into another. The compiler will automatically change one type of data into another if it makes sense. For instance, if you assign an integer value to a floating-point variable, the compiler will convert the int to a float. Casting allows you to make this type conversion explicit, or to force it when it wouldn't normally happen. Type conversion in c can be classified into the following two types:

Implicit Type Conversion:

When the type conversion is performed automatically by the compiler without programmer's intervention, such type of conversion is known as implicit type conversion or type promotion.

```
int x;
for(x=97; x<=122; x++)
{ printf("%c", x); /*Implicit casting from int to char thanks to %c*/
}
```

Explicit Type Conversion:

The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. The explicit type conversion is also known as type casting.

Type casting in c is done in the following form:

(data_type)expression;

Where, data_type is any valid c data type, and expression may be constant, variable or expression.

Operators in C:

C operators are symbols that are used to perform mathematical or logical manipulations. The C programming language is rich with built-in operators. Operators take part in a program for manipulating data and variables and form a part of the mathematical or logical expressions.

C programming language offers various types of operators having different functioning capabilities.

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operator
5. Increment and Decrement Operators
6. Conditional Operator
7. Bitwise Operators

Arithmetic Operators:

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

Relational Operators:

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Logical Operators:

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

Assignment Operator:

An assignment operator is used for assigning a value to a variable. The most common assignment operator is =

Increment and Decrement Operators:

C programming has two operators increment ++ and decrement -- to change the value of an operand (constant or variable) by 1

Conditional Operator:

Conditional operators return one value if condition is true and returns another value is condition is false.

Bitwise Operators:

Bitwise operators are used in C programming to perform bit-level operations.

<u>Applications:</u>

1. Each data type requires different amounts of memory and has some specific operations which can be performed over it.
2. A variable is a named location in a memory where a program can manipulate the data. This location is used to hold the value of the variable and the value of a variable may get change in the program.

Problem Statement 1: Design a program to print the size of fundamental types (Size of Types).

```
#include <stdio.h> int
main()
{
   printf("Size of int: %lu bytes\n", sizeof(int));     printf("Size of
float: %lu bytes\n", sizeof(float));     printf("Size of double: %lu
bytes\n", sizeof(double));     printf("Size of char: %lu byte\n",
sizeof(char));     printf("Size of short: %lu bytes\n",
sizeof(short));     printf("Size of long: %lu bytes\n",
sizeof(long));     printf("Size of long long: %lu bytes\n",
sizeof(long long));     printf("Size of long double: %lu bytes\n",
sizeof(long double));     return 0;
}
```

Output:

Output

```
Size of int: 4 bytes
Size of float: 4 bytes
Size of double: 8 bytes
Size of char: 1 byte
Size of short: 2 bytes
Size of long: 8 bytes
Size of long long: 8 bytes
Size of long double: 16 bytes


=== Code Execution Successful ===
```

<u>Problem Statement 2:</u> Input marks of a student and print "Pass" if marks >= 40 using logical and relational operators.

```
#include <stdio.h> int
main()
{
        int marks;
        printf("Enter marks of the student:");
scanf("%d",&marks);
 //using relational operator (>=) and logical operator (&& for demonstration)   if((marks>=40) &&
(marks<=100))
  {
   printf("Pass\n");
  }
else   {
   printf("Fail\n");
  }
return 0;
}
```

Output:

Output

```
Enter marks of the student:50
Pass


=== Code Execution Successful ===
```

Problem Statement 3: Write a program to demonstrate the increment, decrement, and conditional operators on two integers

```
#include<stdio.h>
int main()
{    int a,b; printf("Enter first
integer (a) : "); scanf("%d",&a);
printf("Enter second integer (b) : ");
scanf("%d",&b);

// Demonstrate increment operator
printf("\n Before increment : a = %d\n",a);
a++;   //increment by 1 printf("\n After
increment : a = %d\n",a);   printf("\n
Before decrement : a = %d\n",b);   b--;
//increment by 1 printf("\n After decrement
: a = %d\n",b);

// demonstrate conditional operator int
max = (a>b) ? a:b;
printf("\n Using conditional operator : \n"); printf("The
greater value between a and b is : %d \n", max); return 0; }
```

Output:

```
C:\Users\Admin\Documents\r

Enter first integer (a): 15
Enter second integer (b): 35

Before increment: a = 15
After increment: a = 16

Before decrement: b = 35
After decrement: b = 34

Using conditional operator:
The greater value between a and b is: 34

--------------------------------
Process exited after 5.81 seconds with return value 0
Press any key to continue . . .
```

Compilation and Execution Steps:

1. Open Dev C++.
2. Click File → New → Source File.
3. Type or paste your C code.
4. Click File → Save As and save the file with a .c extension.
5. Click Execute → Compile (or press **F9**).
6. If there are no errors, click Execute → Run (or press **Ctrl + F10**).
7. View output in the console window.

Practice Question:

Problem Statement 1: Write a C program to input a single character from the user and check whether it is a vowel (a, e, i, o, u) using *relational and logical operators*. Display an appropriate message indicating whether the character is a vowel or not.

Problem Statement 2: Write a C program that inputs the *radius* as an integer and calculates the area of the circle using *type casting* to ensure floating-point division in the calculation. Display the area with two decimal places.
(Hint: Use float or double for the area, and cast radius to float while calculating.)

Problem Statement 3: Write a C program that inputs two integer numbers from the user and demonstrates the use of all arithmetic operators (+, -, *, /, %) on these numbers, displaying each result clearly.

Learning Outcomes: Students will able to
LO2.1 Define the types of data that can be dealt with.
LO2.2 Illustrate the use of variables and constants in C programs.
LO2.3 Illustrate the different types of operators and the various ways in which the operators can be used.

Course Outcome: After completion of course, the students will be able to use primitive data types in computer programming.

Conclusion: In conclusion, the experiment provided a clear understanding of data types, scope, and lifetime of variables in a program, while also emphasizing the importance of type conversions in C and the order of precedence that determines the sequence of operations.

For Faculty Use only

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	Total	Sign
Marks Obtained					